

Longest Common Subsequence

- Given two sequences

$$X = \langle x_1, x_2, \dots, x_m \rangle$$

$$Y = \langle y_1, y_2, \dots, y_n \rangle$$

find a maximum length common subsequence (LCS) of X and Y

- *E.g.:*

$$X = \langle A, B, C, B, D, A, B \rangle$$

- Subsequences of X :

– A subset of elements in the sequence taken in order

$\langle A, B, D \rangle$, $\langle B, C, D, B \rangle$, etc.

Example

$X = \langle A, B, C, B, D, A, B \rangle$

$Y = \langle B, D, C, A, B, A \rangle$

$X = \langle A, B, C, B, D, A, B \rangle$

$Y = \langle B, D, C, A, B, A \rangle$

- $\langle B, C, B, A \rangle$ and $\langle B, D, A, B \rangle$ are longest common subsequences of X and Y (length = 4)
- $\langle B, C, A \rangle$, however is not a LCS of X and Y

Brute-Force Solution

- For every subsequence of X , check whether it's a subsequence of Y
- There are 2^m subsequences of X to check
- Each subsequence takes $\Theta(n)$ time to check
 - scan Y for first letter, from there scan for second, and so on
- Running time: $\Theta(n2^m)$

Making the choice

$X = \langle A, B, D, E \rangle$

$Y = \langle Z, B, E \rangle$

- Choice: include one element into the common sequence (E) and solve the resulting subproblem

$X = \langle A, B, D, G \rangle$

$Y = \langle Z, B, D \rangle$

- Choice: exclude an element from a string and solve the resulting subproblem

Notations

- Given a sequence $X = \langle x_1, x_2, \dots, x_m \rangle$ we define the i -th prefix of X , for $i = 0, 1, 2, \dots, m$

$$X_i = \langle x_1, x_2, \dots, x_i \rangle$$

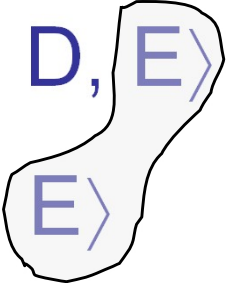
- $c[i, j]$ = the length of a LCS of the sequences

$$X_i = \langle x_1, x_2, \dots, x_i \rangle \text{ and } Y_j = \langle y_1, y_2, \dots, y_j \rangle$$

A Recursive Solution

Case 1: $x_i = y_j$

e.g.: $X_i = \langle A, B, D, E \rangle$
 $Y_j = \langle Z, B, E \rangle$



$$c[i, j] = c[i - 1, j - 1] + 1$$

- Append $x_i = y_j$ to the LCS of X_{i-1} and Y_{j-1}
- Must find a LCS of X_{i-1} and $Y_{j-1} \Rightarrow$ optimal solution to a problem includes optimal solutions to subproblems

A Recursive Solution

Case 2: $x_i \neq y_j$

e.g.: $X_i = \langle A, B, D, G \rangle$

$Y_j = \langle Z, B, D \rangle$

$$c[i, j] = \max \{ c[i - 1, j], c[i, j - 1] \}$$

– Must solve two problems

- find a LCS of X_{i-1} and Y_j : $X_{i-1} = \langle A, B, D \rangle$ and $Y_j = \langle Z, B, D \rangle$
- find a LCS of X_i and Y_{j-1} : $X_i = \langle A, B, D, G \rangle$ and $Y_j = \langle Z, B \rangle$
- Optimal solution to a problem includes optimal solutions to subproblems

Overlapping Subproblems

- To find a LCS of X and Y
 - we may need to find the LCS between X and Y_{n-1} and that of X_{m-1} and Y
 - Both the above subproblems has the subproblem of finding the LCS of X_{m-1} and Y_{n-1}
- Subproblems share subsubproblems

3. Computing the Length of the LCS

$$c[i, j] = \begin{cases} 0 & \text{if } i = 0 \text{ or } j = 0 \\ c[i-1, j-1] + 1 & \text{if } x_i = y_j \\ \max(c[i, j-1], c[i-1, j]) & \text{if } x_i \neq y_j \end{cases}$$

		0	1	2		n	
		y_j	y_1	y_2		y_n	
0	x_i	0	0	0	0	0	
1	x_1	0	→				first
2	x_2	0	→				second
		0			⋮		i
		0					
m	x_m	0	→				
							j

Additional Information

$$c[i, j] = \begin{cases} 0 & \text{if } i, j = 0 \\ c[i-1, j-1] + 1 & \text{if } x_i = y_j \\ \max(c[i, j-1], c[i-1, j]) & \text{if } x_i \neq y_j \end{cases}$$

b & c:

	0	1	2	3	n
y_j :	A	C	D	F	
0 x_i	0	0	0	0	0
1 A	0				
2 B	0				
3 C	0				
m D	0				

j

i

Annotations in the table:
 - An arrow points from the cell (3, C) to the cell (2, D), labeled $c[i, j-1]$.
 - An arrow points from the cell (3, C) to the cell (3, D), labeled $c[i-1, j]$.

A matrix $b[i, j]$:

- For a subproblem $[i, j]$ it tells us what choice was made to obtain the optimal value
- If $x_i = y_j$
 $b[i, j] = \nwarrow$
- Else, if $c[i-1, j] \geq c[i, j-1]$
 $b[i, j] = \uparrow$
- else
 $b[i, j] = \leftarrow$

LCS-LENGTH(X, Y, m, n)

```
1. for i ← 1 to m
2.   do c[i, 0] ← 0
3. for j ← 0 to n
4.   do c[0, j] ← 0
5. for i ← 1 to m
6.   do for j ← 1 to n
7.     do if  $x_i = y_j$ 
8.       then c[i, j] ← c[i - 1, j - 1] + 1
9.         b[i, j] ← "↖"
10.    else if c[i - 1, j] ≥ c[i, j - 1]
11.      then c[i, j] ← c[i - 1, j]
12.        b[i, j] ← "↑"
13.    else c[i, j] ← c[i, j - 1]
14.      b[i, j] ← "←"
15. return c and b
```

The length of the LCS if one of the sequences is empty is zero

Case 1: $x_i = y_j$

Case 2: $x_i \neq y_j$

Running time: $\Theta(mn)$

Example

$X = \langle A, B, C, B, D, A \rangle$

$Y = \langle B, D, C, A, B, A \rangle$

$$c[i, j] = \begin{cases} 0 & \text{if } i = 0 \text{ or } j = 0 \\ c[i-1, j-1] + 1 & \text{if } x_i = y_j \\ \max(c[i, j-1], c[i-1, j]) & \text{if } x_i \neq y_j \end{cases}$$

If $x_i = y_j$

$b[i, j] = \nwarrow$

Else if

$c[i-1, j] \geq c[i, j-1]$

$b[i, j] = \uparrow$

else

$b[i, j] = \leftarrow$

		0	1	2	3	4	5	6
		y_j	B	D	C	A	B	A
0	x_i	0	0	0	0	0	0	0
1	A	0	$\uparrow$ 0	$\uparrow$ 0	$\uparrow$ 0	$\swarrow$ 1	$\leftarrow$ 1	$\swarrow$ 1
2	B	0	$\swarrow$ 1	$\leftarrow$ 1	$\leftarrow$ 1	$\uparrow$ 1	$\swarrow$ 2	$\leftarrow$ 2
3	C	0	$\uparrow$ 1	$\uparrow$ 1	$\swarrow$ 2	$\leftarrow$ 2	$\uparrow$ 2	$\uparrow$ 2
4	B	0	$\swarrow$ 1	$\uparrow$ 1	$\uparrow$ 2	$\uparrow$ 2	$\swarrow$ 3	$\leftarrow$ 3
5	D	0	$\uparrow$ 1	$\swarrow$ 2	$\uparrow$ 2	$\uparrow$ 2	$\uparrow$ 3	$\uparrow$ 3
6	A	0	$\uparrow$ 1	$\uparrow$ 2	$\uparrow$ 2	$\swarrow$ 3	$\uparrow$ 3	$\swarrow$ 4
7	B	0	$\swarrow$ 1	$\uparrow$ 2	$\uparrow$ 2	$\uparrow$ 3	$\swarrow$ 4	$\uparrow$ 4

4. Constructing a LCS

- Start at $b[m, n]$ and follow the arrows
- When we encounter a “↖” in $b[i, j] \Rightarrow x_i = y_j$ is an element of the LCS

		0	1	2	3	4	5	6
		y_j	B	D	C	A	B	A
0	x_i	0	0	0	0	0	0	0
1	A	0	↑ 0	↑ 0	↑ 0	↖ 1	← 1	↖ 1
2	B	0	↖ 1	← 1	← 1	↑ 1	↖ 2	← 2
3	C	0	↑ 1	↑ 1	↖ 2	← 2	↑ 2	↑ 2
4	B	0	↖ 1	↑ 1	↑ 2	↑ 2	↖ 3	← 3
5	D	0	↑ 1	↖ 2	↑ 2	↑ 2	↑ 3	↑ 3
6	A	0	↑ 1	↑ 2	↑ 2	↖ 3	↑ 3	↖ 4
7	B	0	↖ 1	↑ 2	↑ 2	↑ 3	↖ 4	↑ 4

PRINT-LCS(b, X, i, j)

1. **if** $i = 0$ or $j = 0$ Running time: $\Theta(m + n)$
2. **then return**
3. **if** $b[i, j] = \nwarrow$
4. **then** PRINT-LCS($b, X, i - 1, j - 1$)
5. print x_i
6. **elseif** $b[i, j] = \nearrow$
7. **then** PRINT-LCS($b, X, i - 1, j$)
8. **else** PRINT-LCS($b, X, i, j - 1$)

Initial call: PRINT-LCS($b, X, \text{length}[X], \text{length}[Y]$)

Improving the Code

- What can we say about how each entry $c[i, j]$ is computed?
 - It depends only on $c[i - 1, j - 1]$, $c[i - 1, j]$, and $c[i, j - 1]$
 - Eliminate table b and compute in $O(1)$ which of the three values was used to compute $c[i, j]$
 - We save $\Theta(mn)$ space from table b
 - However, we do not asymptotically decrease the auxiliary space requirements: still need table c